Embeddings & Vector Databases

From vector representations and similarity search to practical RAG retrieval

How to use these notes

Read once for the big picture, then revisit the definitions, examples, and comparison tables. The aim is to build intuition first and connect the ideas to practical AI systems.

1. What is an embedding?

An embedding is a list of numbers—a vector—used to represent an item in a form a machine-learning model can process. Items may be words, sentences, documents, images, audio clips, users, or products.

A useful text embedding often places semantically related text closer together in vector space. For example, “How do I reset my password?” and “I forgot my login password” may receive similar vectors even though their words differ.

An embedding is a numerical representation, not the original text and not a complete explanation of its meaning.

2. From text to vectors

Typical pipeline

- Prepare text: normalize or split content as appropriate for the model.
- Send text to an embedding model.
- Receive a fixed-length vector, such as a list of hundreds or thousands of floating-point values.
- Store the vector with an ID and useful metadata.
- At query time, embed the query with a compatible model and compare it with stored vectors.

The exact dimension and values depend on the model. Two different embedding models generally produce vectors that should not be compared directly unless their spaces are specifically compatible.

3. What do vector dimensions mean?

A vector might look like [0.12, -0.08, 0.41, …]. Each coordinate is a learned feature in a high-dimensional representation. Usually, a person cannot assign a simple human-readable meaning to each individual coordinate.

The overall geometry matters more than any one coordinate. During training, the model learns representations that help with a task such as semantic similarity, retrieval, or classification.

4. Similarity and distance

Cosine similarity

Cosine similarity compares the angle between two vectors. It is commonly used for text embeddings because it emphasizes direction rather than vector magnitude.

```
cosine(A, B) = (A · B) / (||A|| × ||B||)
```

Dot product and Euclidean distance

The dot product can be useful when vector magnitudes carry meaning or when a model is trained for that scoring rule. Euclidean distance measures straight-line distance between points. Use the metric recommended for the chosen embedding model and database configuration.

Do not assume that every vector database ranks results using cosine similarity by default. Check the index and metric configuration.

5. What is a vector database?

A vector database stores vectors and supports similarity search. It often also stores IDs, text chunks, and metadata, and provides filtering, indexing, update, and deletion operations.

A conventional relational database is strong at structured queries such as “orders placed after a date.” A vector database is designed to find items whose vectors are close to a query vector. Many applications use both.

Common capabilities

- Store and retrieve vectors by ID.
- Nearest-neighbor or approximate-nearest-neighbor search.
- Metadata filters, such as document type, access level, or date.
- Persistence, indexing, updates, and deletion.
- Hybrid retrieval that combines lexical search and vector similarity, depending on the product.

6. How vector search works

- Create embeddings for documents or chunks and store them.
- Convert a user query into a vector using the same compatible embedding model.
- Search the index for the nearest vectors, optionally applying metadata filters.
- Return the top-k matches with IDs, scores, and any stored payload or metadata.
- Use the retrieved content in the application, such as showing search results or providing context to an LLM.

Many systems use approximate nearest-neighbor indexes to search quickly across large collections. Approximate search trades some recall or exactness for speed and lower resource use.

7. What is stored in a vector database?

A practical record may include:

```
id: "policy_chunk_042"
vector: [0.12, -0.08, 0.41, ...]
text: "Employees may request leave..."
metadata: {"source": "handbook.pdf", "page": 8}
```

The vector supports similarity search. The text is the content to return. Metadata supports filtering, attribution, and traceability. Some systems keep full documents elsewhere and store only references or chunks in the vector database.

8. Embeddings in a RAG system

Retrieval-Augmented Generation (RAG) retrieves relevant external content and supplies it to a generative model as context. Embeddings commonly support the retrieval stage.

Indexing time

- Collect source documents.
- Split documents into chunks that are useful for retrieval.
- Generate an embedding for each chunk.
- Store vectors, chunk text or references, and metadata.

Query time

- Embed the user's question.
- Retrieve the most relevant chunks.
- Optionally rerank results and remove duplicates.
- Pass selected chunks to the LLM with the user's question.
- Generate an answer, ideally with citations to the retrieved sources.

The vector database retrieves candidate evidence; the LLM generates a response. Retrieval quality does not guarantee that the final answer is correct.

9. Chunking and metadata

Large documents are often split into smaller chunks because an entire document embedding can blur several topics together. Chunk size, overlap, headings, and document structure affect retrieval quality.

Very small chunks can lose context; very large chunks can dilute relevance and use more context-window space. Evaluate chunking on representative questions instead of choosing a size by habit.

Metadata can include source, page, title, tenant, timestamp, language, or permissions. Apply access-control filters before returning content; similarity alone is not a security boundary.

10. Vector database vs. relational database vs. keyword search

Vector database

Best suited to nearest-neighbor search over dense vector representations, often with metadata filtering.

Relational database

Best suited to structured records, joins, transactions, constraints, and exact filtering. Some relational systems also support vector extensions.

Keyword / lexical search

Matches terms and lexical signals. It can be excellent for exact names, IDs, rare terms, and phrases. It may miss a paraphrase that uses different words.

Hybrid retrieval combines lexical and vector signals to balance semantic matching with exact-term matching.

11. Choosing and evaluating a system

- Embedding model: language support, domain quality, vector dimensions, latency, cost, and license.
- Database: scale, filtering, persistence, latency, operational burden, deployment, and access control.
- Retrieval: measure whether relevant documents appear in the top-k results.
- Generation: separately assess factual correctness, grounding, citation quality, and refusal behavior.

Useful retrieval metrics include Recall@k (whether relevant items appear among the top k), Precision@k (how many top-k items are relevant), and mean reciprocal rank (how highly the first relevant result appears). Test on real questions and known relevant documents.

12. Common failure modes

- Poor chunk boundaries split an answer across chunks or combine unrelated topics.
- Embedding a query with a different or incompatible model produces unreliable comparisons.
- A similarity score is not automatically a calibrated probability of relevance.
- Stale or duplicated documents can crowd out useful results.
- Missing metadata filters can return the wrong version, tenant, or access level.
- A fluent LLM may still make unsupported claims if retrieval misses the evidence or the prompt is weak.

13. A small worked example

Suppose a support knowledge base contains three chunks: (A) “Reset your account password,” (B) “How to update a billing address,” and (C) “Recover access when you forget your login.” A query says, “I can’t remember my password.”

Keyword search may match A strongly because it shares “password.” A semantic embedding may rank A and C highly because both discuss password recovery, even if the exact wording differs. A hybrid system can combine both kinds of evidence.

The application can then send the best chunks to an LLM and ask it to answer only from the retrieved support material.

14. Check your understanding

- What is the difference between an embedding and a vector database?
- Why should query and document vectors come from compatible embedding models?
- When might keyword search outperform vector search?
- Why are chunking and metadata important in RAG?
- What does Recall@k measure?
- Why is a similarity score not the same as a probability?

Suggested learning order

First understand vectors and cosine similarity, then embedding models, then nearest-neighbor search and metadata filtering, and finally the RAG indexing and query pipelines.